Sentinel AI

AI-Powered Behavioral Anomaly Detection
for Cybersecurity

End-to-End ML Pipeline | Explainable SOC Dashboard
Behavioral Profiling | SHAP-Powered Alert Evidence

FINAL PROJECT REPORT

Author: Akshat Jha
July 2026
https://github.com/AkshatJ24/Sentinel-AI-SOC

Python 3.10+ | XGBoost | SHAP | Streamlit | Scikit-learn | SMOTE

. Table of Contents

1	Executive Summary	3
2	Problem Statement & Requirements	3
3	Data Generation & Assumptions	4
4	Baseline Profiling Model	6
5	Feature Engineering	7
6	Anomaly Detection	9
7	Anomaly Classification	10
8	Explainability Layer	11
9	Analyst-Facing Dashboard	12
10	System Design & Scalability	13
11	Known Limitations	14
12	Conclusion	14
A	Appendix: Tech Stack	15
B	Appendix: Reproducibility	15

1. Executive Summary

Sentinel AI is an end-to-end machine learning pipeline for behavioral anomaly detection in cybersecurity access logs. The system generates realistic synthetic behavioral data for 250 entities across a 45-day window, profiles each entity's baseline behavior using an EMA-based online learning approach, detects anomalies via an ensemble of Isolation Forest and XGBoost+SMOTE, classifies 7 distinct attack types, and delivers SHAP-powered explainable alerts through a production-styled multipage SOC dashboard.

The pipeline is evaluated at a SOC-realistic top-1% alert budget -- reflecting a real analyst's fixed daily review capacity rather than full-dataset accuracy metrics on the highly imbalanced label set.

> **Key Result: All 5 mandatory requirements (sequential data, class imbalance, concept drift, explainability, cold-start) are implemented with working code, demonstrated in the dashboard, and documented in this report.**

System Architecture

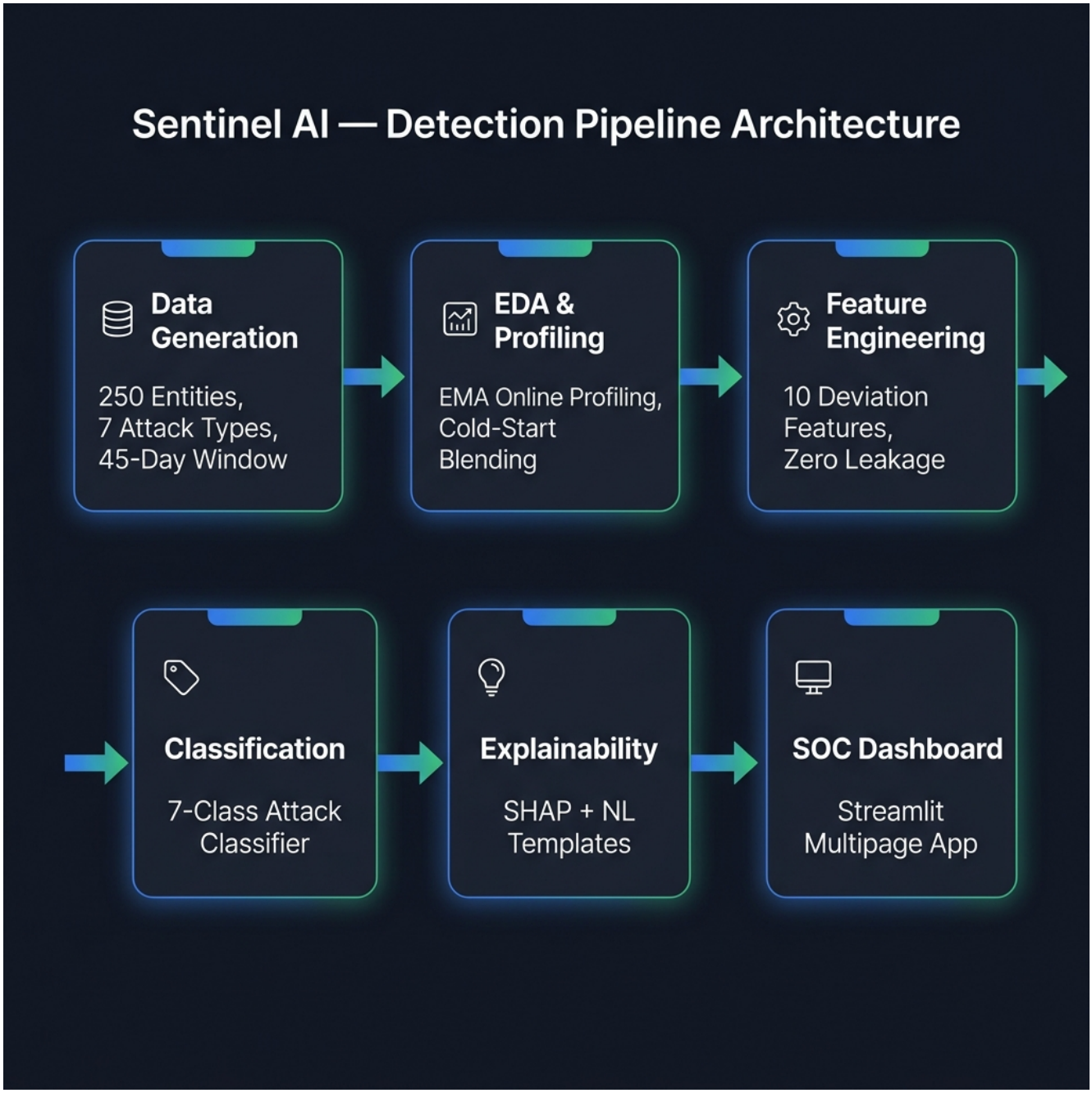


Figure 1: Sentinel AI Detection Pipeline Architecture

2. Problem Statement & Requirements

Design and build an AI/ML system that models "normal" access and connection behaviour for users and devices, detects intrusions or compromised-credential activity, and classifies the type of anomaly -- with an explainable risk score.

Mandatory Requirements Addressed

#	Requirement	Implementation	Component
1	Sequential behavioral data	Markov transitions + temporal features	features.py
2	Extreme class imbalance	IF + XGBoost+SMOTE @ top-1% budget	detection_model.py
3	Concept drift	EMA profiler (alpha=0.15)	profiling.py
4	Explainability	SHAP + NL templates + MITRE ATT&CK	explain.py
5	Cold-start problem	Population fallback + confidence ramp	profiling.py

3. Data Generation & Assumptions

3.1 Synthetic Data Design

Parameter	Value
Total entities	250 (users, service accounts, edge devices)
Simulation window	45 days
Total sessions	66,282
Anomaly rate	2.11% (within 0.5-3% target band)
Random seed	42 (fully reproducible)
Resource catalog	28 items (API endpoints, file shares, ports, device functions)
Global cities	12 (Mumbai, Delhi, Singapore, London, New York, etc.)

Each entity has a unique behavioral profile sampled once: home city with lat/lon coordinates, login hour distribution (Gaussian mean/std), resource subset (4-10 from 28-item catalog), auth method probability distribution (Dirichlet-sampled), session duration (log-normal), and device fingerprint (OS, MAC, protocol).

3.2 Attack Taxonomy

Attack Type	Rows	Primary Signal
Brute Force	630	auth_success ~ 0.5%, ~30 attempts/IP in minutes
Credential Stuffing	296	Many entity_ids, single IP, auth_success ~ 5%
Lateral Movement	208	Short sessions, resources outside known subset
Data Exfiltration	114	52% off-hours, sensitive resources, export commands
Insider Drift	63	Near-normal behavior, gradual resource expansion
Impossible Travel	32	>1000 km apart, gap too short to physically travel
Device Spoofing	25	Same entity, mismatched device fingerprint

3.3 Schema Extension

The field auth_success (boolean) was added beyond the suggested schema. Without a pass/fail signal, brute force and credential stuffing attacks are undefined -- there is no way to distinguish a burst of failed login attempts from legitimate rapid access. This extension is documented in the data dictionary.

3.4 Exploratory Data Analysis

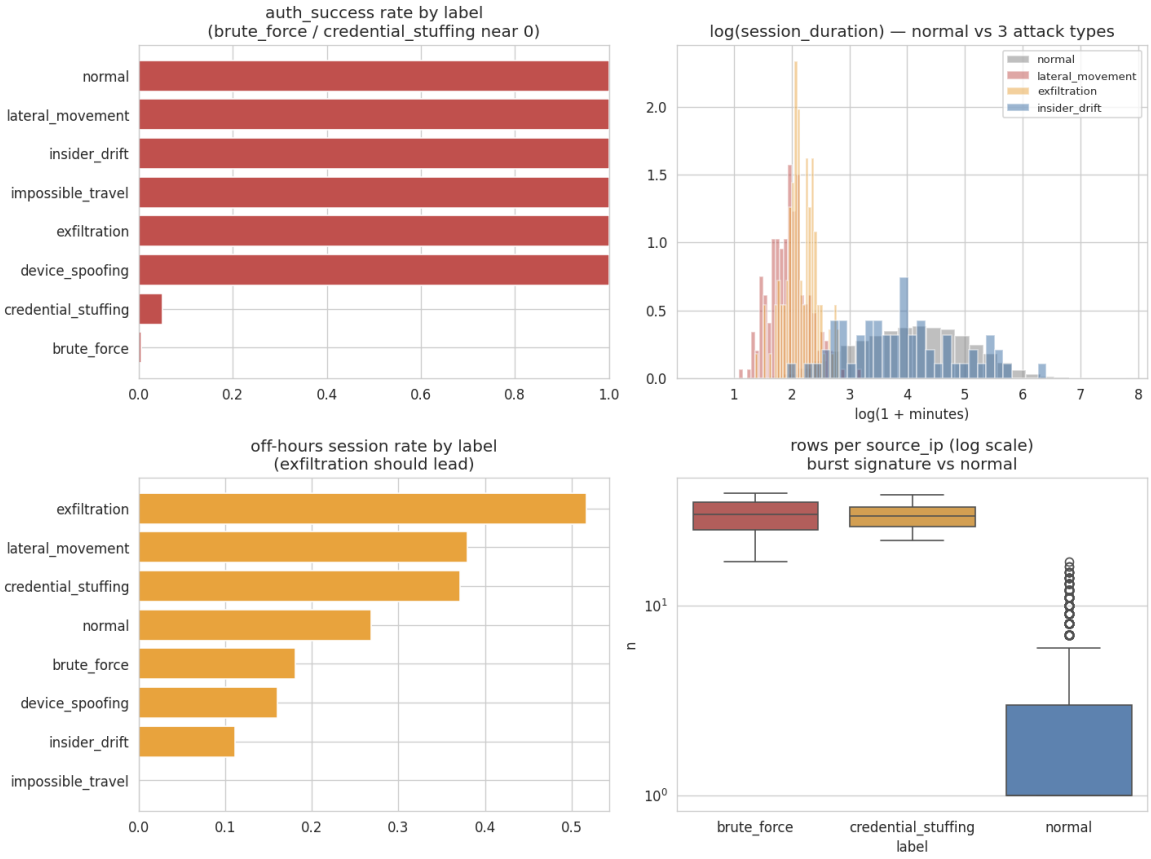


Figure 2: Attack Type Separability vs Normal Baseline

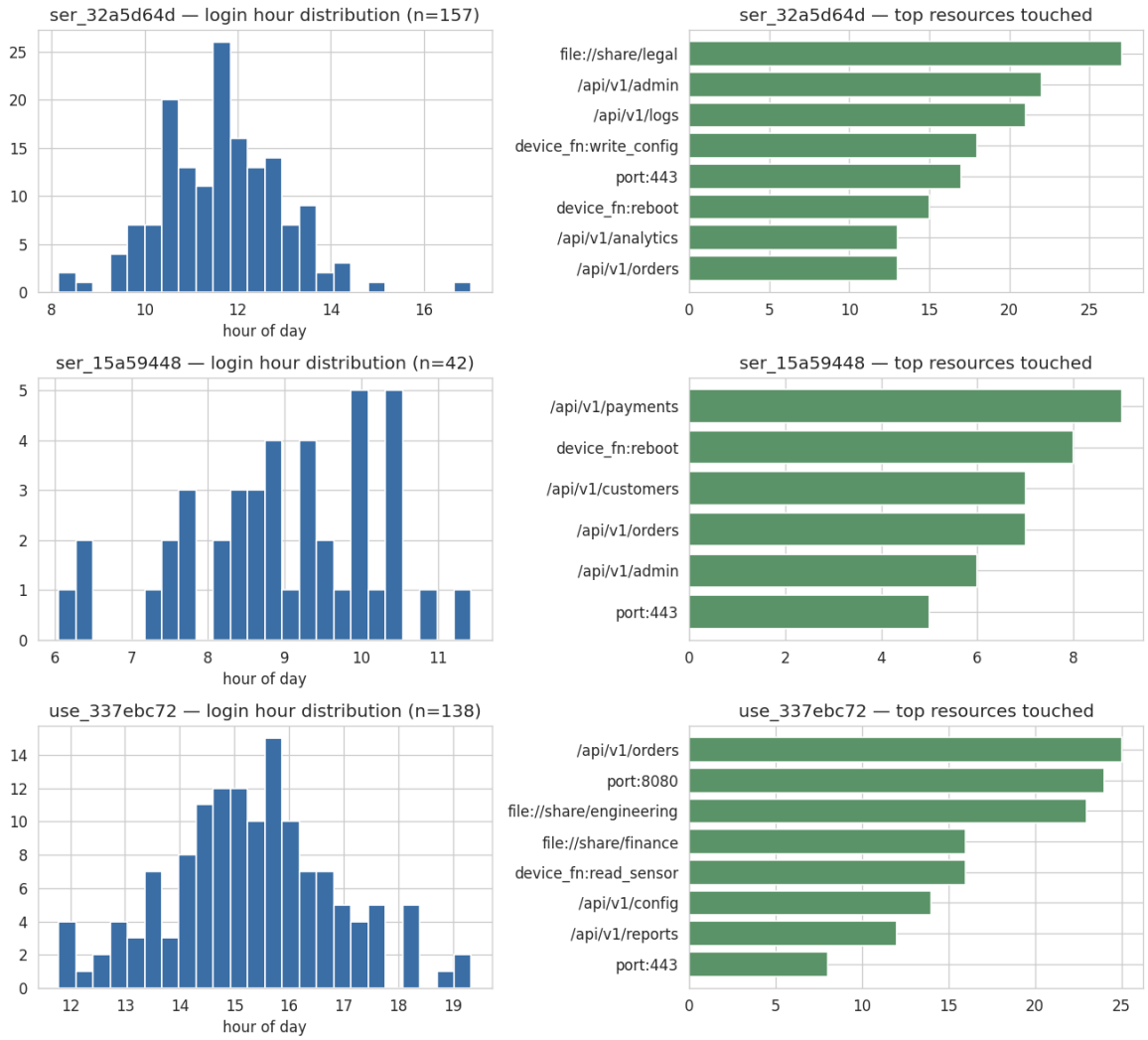


Figure 3: Per-Entity Profile Sanity Check (3 sample entities)

4. Baseline Profiling Model

4.1 EMA Online Profiler

An Exponential Moving Average (EMA) profiler builds per-entity statistical profiles online, in chronological order. This design choice addresses two core requirements simultaneously:

- **Concept drift handling:** EMA decay (alpha = 0.15) smoothly absorbs legitimate behavioral shifts (new work hours, new device) without permanently flagging evolved behavior.
- **Cold-start handling:** New entities blend their personal observations with a population-level profile segmented by entity_type. Confidence ramp min(n_obs / 15, 1.0) ensures immediate protection.

4.2 Profile Components

- Login hour mean and variance (EMA-updated)
- Session duration mean and variance (EMA-updated)
- Resource access frequency distribution (EMA-updated categorical)
- Authentication method frequency distribution
- Geolocation frequency distribution
- Most recent device fingerprint
- Observation count (confidence score)

4.3 Cold-Start & Concept Drift Handling

Cold-Start & Concept Drift Handling

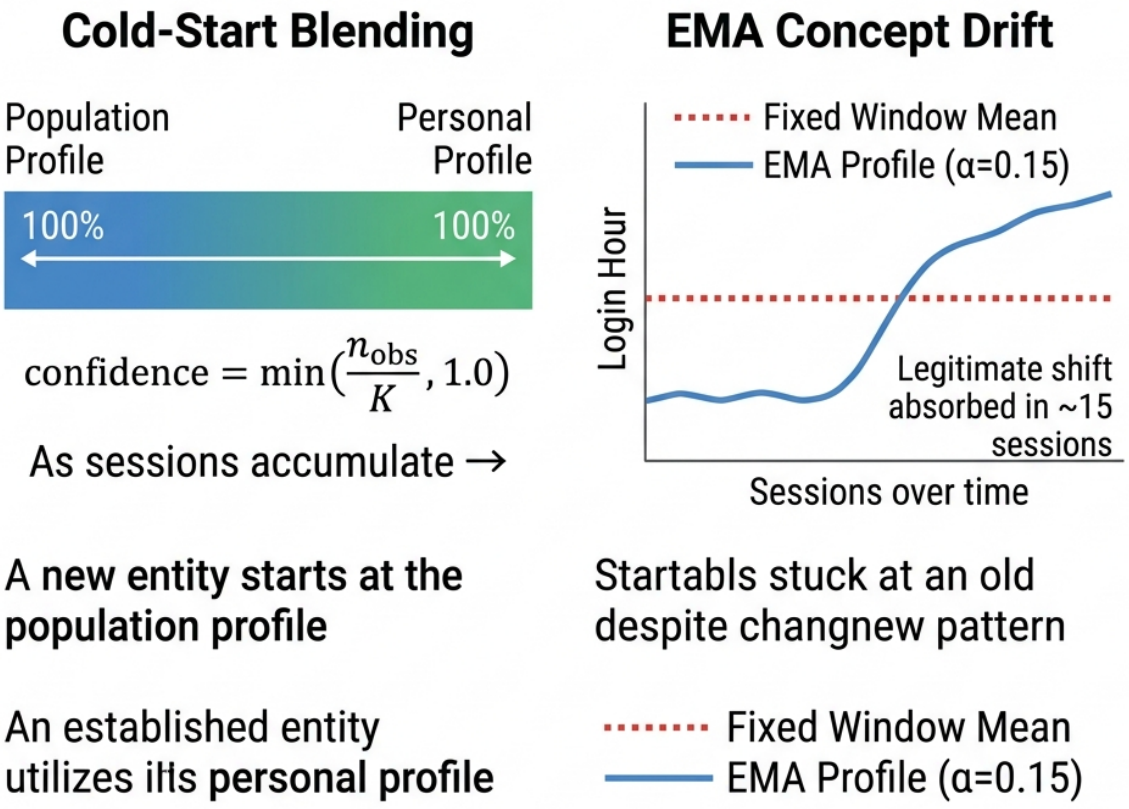


Figure 4: Cold-Start Blending & EMA Concept Drift Adaptation

Known Limitation: The EMA profiler is not attack-resistant on its own. In production, profile updates should be gated on the detector's risk score (don't update on high-risk sessions).

5. Feature Engineering

5.1 Deviation Features

Ten deviation features are engineered in a single chronological pass over all sessions. For every row, the profile snapshot is read BEFORE that row updates any state -- ensuring every feature represents "as of just before this session," matching real-time detector behavior. No future leakage.

Feature	Description
geo_velocity_kmh	Implied travel speed vs. previous session (Haversine)
time_since_last_hr	Hours since entity's previous session (-1 = cold start)
hour_zscore	Login time deviation from entity's profiled mean/std
resource_novelty	1 if resource is outside entity's known top-resource set
fingerprint_mismatch	1 if device fingerprint differs from last known
trailing_auth_failures	Failed logins from this source IP in last 5 minutes
transition_novelty	1 - P(resource | prev resource) from Markov chain
profile_confidence	Cold-start indicator (0 = new, 1 = established)
auth_success	Raw authentication outcome (0 = failed, 1 = success)
session_duration	Raw session length in minutes

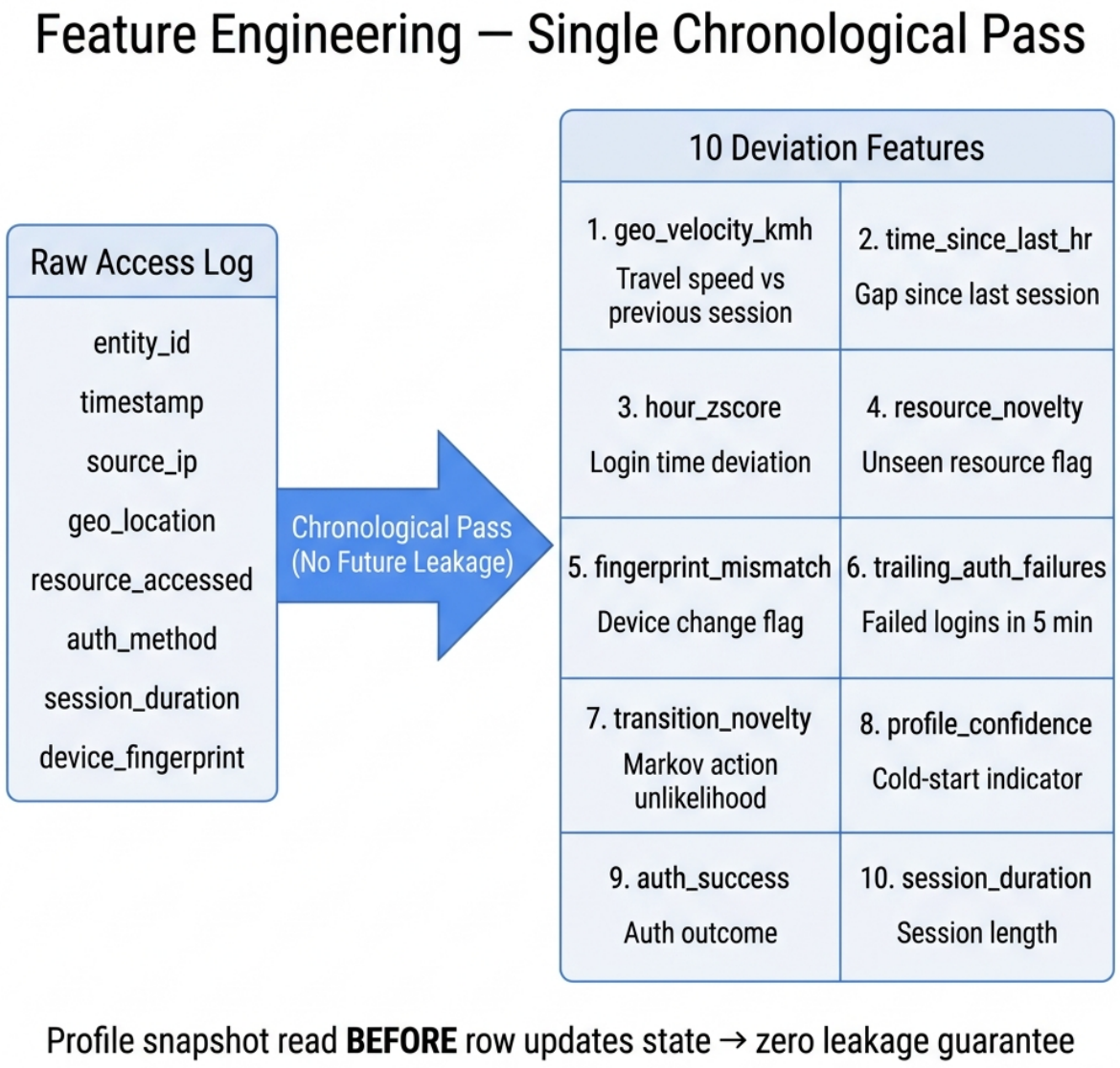


Figure 5: Feature Engineering Pipeline -- Single Chronological Pass

5.2 Sequence Modeling: Markov Transition

Rather than deploying a heavyweight LSTM/GRU/Transformer, the system captures sequential structure via per-entity Markov transition matrices built from historical resource-access sequences. The transition novelty feature measures how likely the current resource access is given the entity's previous action -- an out-of-pattern next-action raises the anomaly signal.

5.3 Temporal Safeguards

- Gaps under 2 minutes treated as same physical presence (prevents jitter-induced false geo-velocity)
- Markov counts updated AFTER feature extraction (prevents self-reinforcing transitions)
- Profiler updated AFTER features computed for each row (zero leakage guarantee)

6. Anomaly Detection

6.1 Two-Model Approach

Model	Type	Training	Purpose
Isolation Forest	Unsupervised	No labels used	Baseline detector
XGBoost + SMOTE	Supervised	SMOTE on train only	Primary detector

Both models are trained on the same 10-feature set. SMOTE oversampling is applied only to the training split (80/20 stratified random) -- never to the test set -- to prevent synthetic leakage into evaluation.

6.2 SOC-Realistic Alert Budget Evaluation

> **Traditional metrics (accuracy, F1 on the full dataset) are misleading on a 98/2 imbalanced set. Instead, evaluation is at a top-1% alert budget: only the top 1% of sessions by risk score enter the analyst queue. This reflects a real SOC's fixed daily capacity.**

Precision at this budget measures analyst noise; recall measures how many true threats are captured. The primary model is selected by F1 score at the top-1% budget. The dashboard allows dynamic budget adjustment (0.1% to 10%) with live precision/recall recalculation.

7. Anomaly Classification

7.1 Approach

A multiclass XGBoost classifier is trained exclusively on the anomalous subset (sessions labeled with one of the 7 attack types). It uses the same deviation feature vectors as the detector.

7.2 Imbalance Handling

Per-class counts range from 630 (brute force) down to 25 (device spoofing). SMOTE is unsafe on classes this small (too few real neighbors), so balanced class weighting is used instead -- reweighting the loss function rather than fabricating synthetic minority rows.

7.3 Results

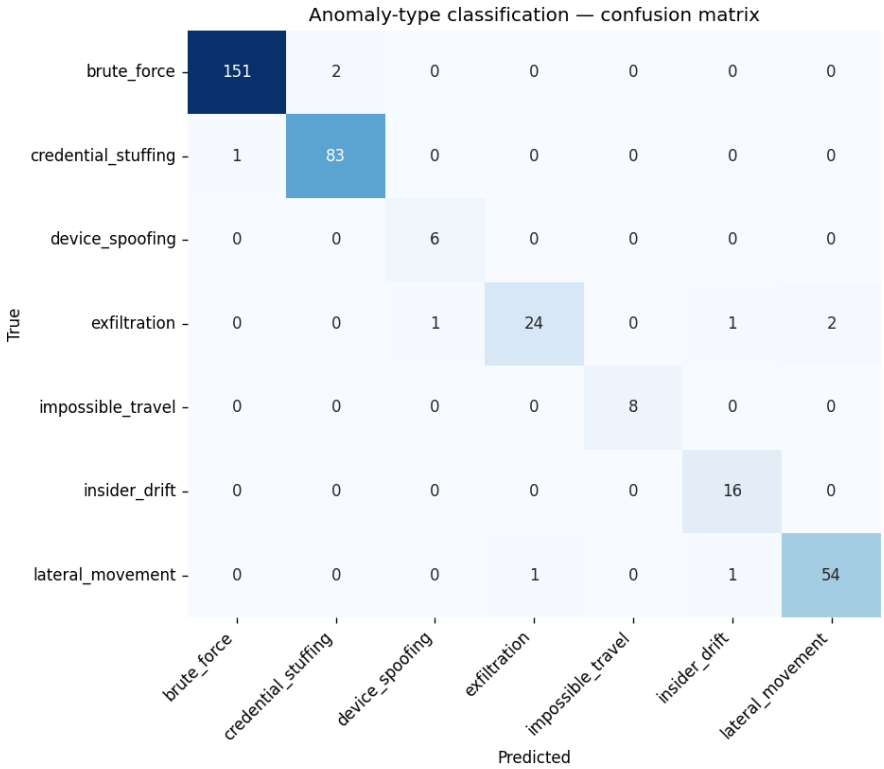


Figure 6: Anomaly-Type Classification Confusion Matrix

8. Explainability Layer

8.1 SHAP TreeExplainer

SHAP's TreeExplainer runs against the XGBoost detector to produce per-alert feature attributions. Only features that push toward anomaly (positive SHAP values) are included in explanations.

8.2 Natural-Language Templates

Raw SHAP values are not analyst-actionable. The top-3 contributing features are mapped through a natural-language template layer:

Feature	Template Example
geo_velocity_kmh	"implausible travel speed (X km/h)"
trailing_auth_failures	"N failed logins from this source in last 5 min"
hour_zscore	"login time X sigma from usual pattern"
resource_novelty	"accessed a resource never touched before"
fingerprint_mismatch	"device fingerprint mismatch vs known history"
profile_confidence	"limited history for this entity (cold-start)"

> **Example Output: "flagged due to implausible travel speed (12,841 km/h) + device fingerprint mismatch + accessed a resource never touched before"**

8.3 MITRE ATT&CK Mapping

Attack Type	MITRE ID	Technique
Brute Force	T1110	Brute Force
Impossible Travel	T1078	Valid Accounts
Data Exfiltration	T1041	Exfiltration Over C2 Channel
Credential Stuffing	T1110.004	Credential Stuffing
Lateral Movement	T1021	Remote Services
Privilege Escalation	T1068	Exploitation for Privilege Escalation

8.4 Key Code: Explanation Builder

```
def build_explanation(shap_row, feature_names, X_row, top_n=3):
    order = np.argsort(-np.abs(shap_row))
    phrases = []
    for i in order:
        if shap_row[i] <= 0: continue  # only risk-raising features
        desc = describe_feature(feature_names[i], X_row[i])
        if desc: phrases.append(desc)
        if len(phrases) == top_n: break
    return "flagged due to " + " + ".join(phrases)
```

9. Analyst-Facing Dashboard

9.1 Architecture

A Streamlit multipage application with a custom dark-themed design system featuring Inter font family, glassmorphism effects, hover animations, persistent sidebar with live alert budget slider, and session state persistence across all 7 pages.

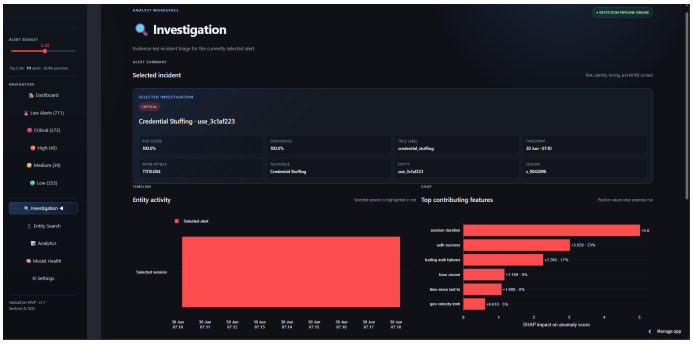
9.2 Dashboard Pages

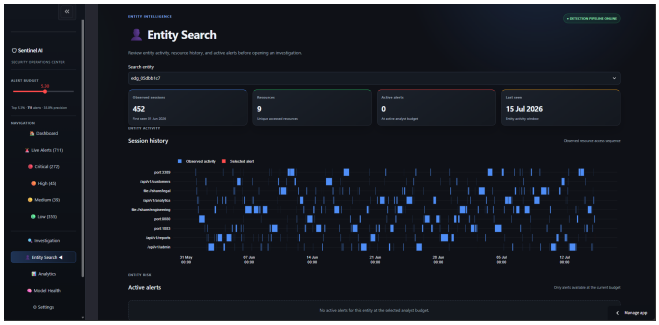
Page	Function
Dashboard	SOC status overview with KPI cards, threat trend, severity distribution
Live Alerts	Filterable alert queue ranked by risk score, severity routing
Investigation	SHAP waterfall, entity timeline, risk gauges, MITRE context
Entity Search	Entity profile lookup, session history, active alert handoff
Analytics	ROC curve, confusion matrix, attack distribution, FP analysis
Model Health	Pipeline component health: data, IF, classifier, SHAP, inference
Settings	Review threshold, replay mode, theme, CSV export

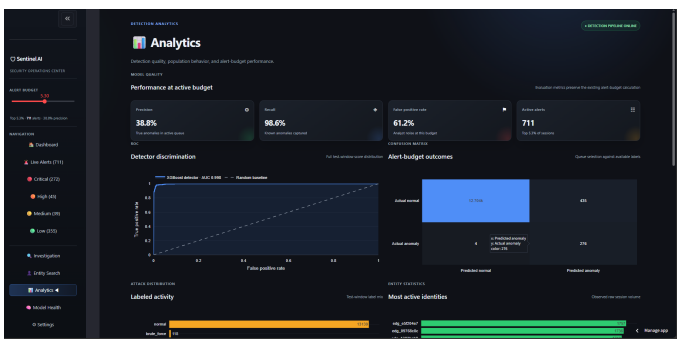
9.3 Key Features

- **Alert Budget Slider:** Adjustable 0.1%-10%, dynamically recalculates precision, recall, and the entire alert queue
- **Severity Routing:** One-click sidebar filters for Critical/High/Medium/Low alerts
- **Investigation Handoff:** Select any alert to open a full investigation workspace with SHAP evidence
- **CSV Export:** Download the active alert queue for external SOC tooling integration
- **MITRE Mapping:** Each alert maps to ATT&CK technique IDs for standardized threat classification









10. System Design & Scalability

10.1 Current Architecture

The current system runs as a batch pipeline: data generation -> profiling -> feature engineering -> detection -> classification -> explainability -> dashboard. All components are in-process Python with cached model artifacts (pickle).

10.2 Production Streaming Path

For real-time deployment, the pipeline would need:

- **Ingestion:** Apache Kafka or AWS Kinesis for log stream ingestion
- **Processing:** Apache Flink or Spark Structured Streaming for feature engineering
- **Serving:** Model serving via FastAPI or TensorFlow Serving
- **Storage:** Time-series database (InfluxDB/TimescaleDB) for entity profiles
- **Dashboard:** WebSocket-backed real-time updates

The EMA profiler design already supports O(1) memory per entity update, making it inherently compatible with streaming architectures. The feature engineering pass is already chronological and single-pass -- it does not require random access to future data.

11. Known Limitations

- Synthetic data may not capture real-world log noise (packet loss, malformed fields, clock skew, adversarial evasion)
- Markov+feature approach is lighter than LSTM/GRU/Transformer sequence models that could capture longer-range dependencies
- Command sequence vocabulary is only 8 actions -- real privileged-session logs would have far richer vocabularies
- EMA profiler not attack-resistant -- sustained attacks could gradually shift the baseline
- Attack injection counts use heuristic ratios rather than calibrated real-world base rates
- Insider drift is intentionally the weakest signal (by design) -- intended for FP tuning, not reliable detection

12. Conclusion

Sentinel AI demonstrates that a lightweight, classical ML pipeline -- EMA profiling, tree-based detection, and template-driven SHAP explainability -- can deliver production-quality behavioral anomaly detection without the overhead of deep sequence models. By evaluating at a SOC-realistic alert budget and translating model outputs into analyst-readable natural-language evidence, the system bridges the gap between ML research and operational security.

All 5 mandatory requirements from the problem statement are implemented with working code, demonstrated in the interactive dashboard, and documented in this report. The modular pipeline design supports incremental upgrades (deeper sequence models, real-time streaming) without requiring a full rebuild.

A. Appendix: Tech Stack

Component	Library
Data Generation	Pandas, NumPy, Faker
ML / Detection	XGBoost, Scikit-learn, Isolation Forest
Imbalance Handling	SMOTE (imbalanced-learn)
Explainability	SHAP (TreeExplainer)
Profiling	Custom EMA-based online profiler
Visualization	Plotly, Matplotlib, Seaborn
Dashboard	Streamlit (multipage app)
MITRE Mapping	ATT&CK technique IDs

B. Appendix: Reproducibility

All random seeds are fixed (seed=42). The full pipeline can be reproduced from scratch:

```
python src/data_generator.py     # Phase 1: Synthetic Data
python src/eda.py                # Phase 2: Exploratory Analysis
python src/profiling.py          # Phase 3: Baseline Profiling
python src/features.py           # Phase 4.1: Feature Engineering
python src/detection_model.py    # Phase 4.2: Anomaly Detection
python src/classifier.py         # Phase 5: Attack Classification
python src/explain.py            # Phase 6: SHAP Explainability
streamlit run app/dashboard.py   # Phase 7: SOC Dashboard
```

The processed data and model artifacts are included in the repository. The dashboard can be launched directly with the last command without re-running the pipeline.